

Domain-Specific Text-to-SQL Fine-Tuning

QLoRA Fine-Tuning of Qwen2.5-Coder-1.5B for Fintech SQL Generation

Engineering Report | Richardy Lobo Sapan | June 2026

1. Executive Summary

This report documents the design, implementation, and evaluation of a domain-specific Text-to-SQL fine-tuning pipeline, built to convert natural language questions into executable SQL queries for a fintech database context.

The objective was to fine-tune a small language model ($\leq 3B$ parameters) using parameter-efficient methods on free-tier compute, then evaluate the resulting model strictly on real-world execution against a live SQLite database, not on textual similarity to reference answers.

The technical stack consisted of **Qwen2.5-Coder-1.5B-Instruct** as the base model, fine-tuned via **QLoRA** (4-bit quantization + LoRA adapters, rank=16, alpha=32, learning rate $2e-4$) using **Unsloth** for memory-efficient training, on a single free-tier Google Colab Tesla T4 GPU.

Two infrastructure-level framework bugs were encountered and resolved during training, and the evaluation phase produced a result that, on the surface, looks like total failure (0% execution accuracy for both the base and fine-tuned model) but on closer inspection reveals two distinct and instructive failure modes. Both the engineering recovery process and the error analysis are detailed below, as they were the most technically substantive part of this project.

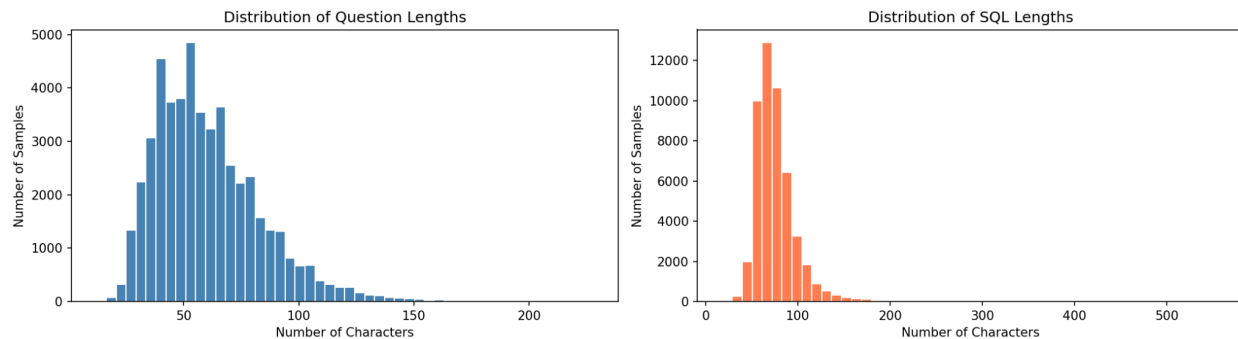
2. Architecture & Training Strategy

2.1 Model and Method Selection

Qwen2.5-Coder-1.5B-Instruct was selected as the base model for three reasons: (1) it is pre-trained on code and SQL, meaning fine-tuning only needs to teach schema-grounding and prompt-format discipline rather than SQL syntax from scratch; (2) at 1.5B parameters it comfortably satisfies the $\leq 3B$ constraint while leaving headroom on a 16GB T4; and (3) the Instruct variant already follows structured prompt formats reliably, which reduces the data volume needed to teach a new formatting convention.

QLoRA was the natural fit for the compute budget. Full fine-tuning of 1.5B parameters is not feasible on a free-tier T4. QLoRA quantizes the frozen base model to 4-bit and trains only small low-rank adapter matrices injected into the attention and MLP projection layers, reducing trainable parameters to roughly 1.2% of the full model, confirmed at runtime as 18,464,768 trainable parameters out of 1,562,179,072 total.

2.2 Dataset Preparation



The training set used was **b-mc2/sql-create-context** (~45,000 examples after sampling and filtering), substituted in after the originally planned dataset, *seeklhy/SynSQL-2.5M*, proved impractical: it is distributed as a single ~9.4GB unsharded file with no native slicing support, making partial download infeasible on bandwidth-constrained connections. *sql-create-context* provided the same (question, schema, SQL) triple structure at a fraction of the size, with no loss of structural fidelity for this task.

Each example was mapped into a strict, consistent template:

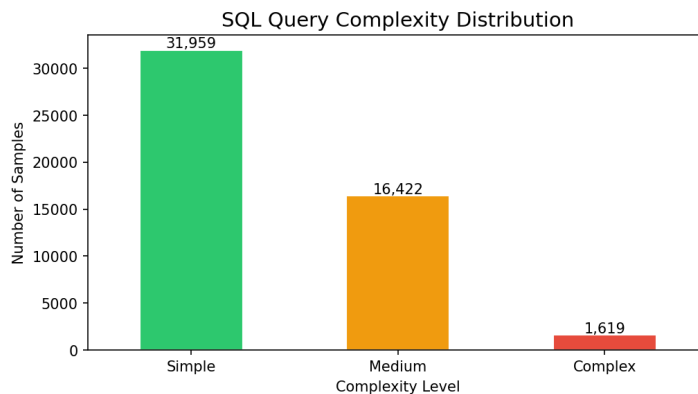
```
### Question:
{question}

### Schema:
{schema}

### SQL:
{sql}
```

This formatting choice was deliberate. The goal was to strip away conversational instruction-following behavior and condition the model to act as a pure, deterministic SQL generator, input in, single SQL statement out, nothing else. As detailed in Section 4, this objective was achieved structurally but introduced a different failure mode.

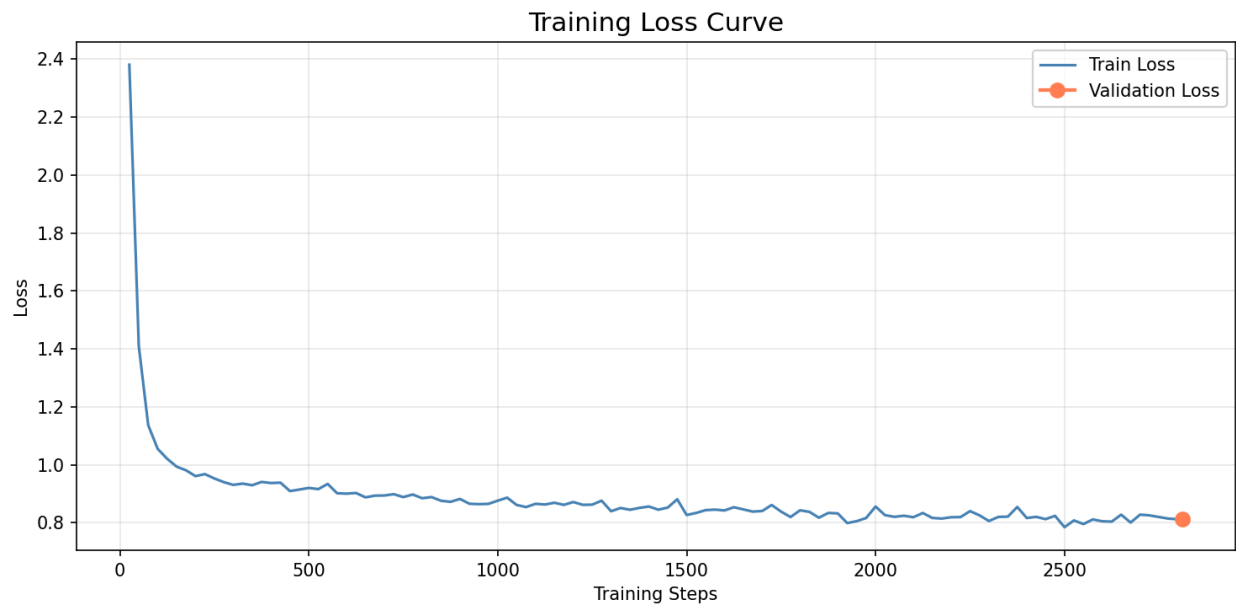
Exploratory data analysis was performed on the sampled set prior to formatting, covering question/SQL length distributions and a keyword-based SQL complexity classification (Simple / Medium / Complex), to confirm the sample was not skewed toward trivial single-table queries before training began.



2.3 Training Configuration

Hyperparameter	Value
Base model	Qwen2.5-Coder-1.5B-Instruct (4-bit)
Method	QLoRA via Unsloth
LoRA rank (r)	16
LoRA alpha	32
Target modules	q/k/v/o_proj, gate/up/down_proj
Learning rate	2e-4, cosine schedule
Effective batch size	16 (per-device 4 × grad-accum 4)
Max sequence length	2048 tokens
Optimizer	AdamW (8-bit)
Epochs (planned / actual)	3 / 1

Note on epoch count: the original training plan specified 3 epochs. Due to the framework-level checkpointing failure described in Section 2.4, the actual completed run was reduced to 1 epoch to secure a usable adapter within the project's time budget. This is a direct, traceable consequence of the infrastructure issue below, not an independent modeling decision, and it is the most likely contributor to the under-convergence discussed in Section 4.



2.4 Framework Challenges and Engineering Resolution

Two distinct framework-level failures were encountered during training and resolved through direct intervention rather than by restarting blindly.

Issue 1: Checkpoint Serialization Failure (PicklingError)

During the initial 3-epoch run, training progressed normally through step 500 (training loss 0.940, validation loss 0.922) before crashing during checkpoint serialization with: `PicklingError: Can't pickle <class 'trl.trainer.sft_config.SFTConfig'>: it's not the same object as trl.trainer.sft_config.SFTConfig`

Diagnosis: this is a class-identity collision, not a data or model issue. The same class had been imported under two different module resolution paths in memory, almost certainly a residual effect of resolving an earlier pip dependency conflict between unsloth-zoo's required transformers/trl/peft version ranges and an initial set of pinned versions that proved too old. The LoRA adapter weights for that step had been written successfully to disk before the crash (confirmed via identical file sizes between the partial checkpoint and a later emergency save, ~73.9MB), but the `trainer_state.json` required for a clean resume was never finalized, making a standard `resume_from_checkpoint` path unrecoverable.

Fix applied: rather than continuing to patch the serialization path under time pressure, checkpointing was disabled entirely (`save_strategy="no"`), and a single continuous training pass was run to completion, with the LoRA adapter saved directly via `model.save_pretrained()` at the end rather than relying on the trainer's intermediate checkpoint mechanism. This traded crash-recovery safety for a model class with no known serialization defect, given the time constraint.

Issue 2: Adapter Merge Corruption

After securing the adapter, `model.merge_and_unload()` was used to fuse the LoRA weights into the base model for a simpler, dependency-free inference path. This operation is destructive and in-place: it mutates the live model object. Running a sanity-check generation against the model object immediately afterward produced incoherent, multilingual token noise rather than SQL, a strong indicator that merging LoRA deltas into a 4-bit-quantized base produced a numerically corrupted weight set, a known incompatibility between certain Unsloth/PEFT/4-bit version combinations.

Fix applied: the merged-model path was abandoned. The inference pipeline was redesigned to load the unmodified base model fresh and apply the saved LoRA adapter weights dynamically at inference time, which is both the safer and the more standard pattern for LoRA deployment. This was verified to produce coherent SQL-shaped output, confirming the saved adapter itself was never corrupted, only the merge operation was.

3. Evaluation Methodology

The evaluation was deliberately designed around execution accuracy rather than textual or token-level similarity to a reference SQL string. This decision reflects how the original task framed success: a generated query's value is whether it can actually run against a real schema and return a result, not whether its tokens overlap with a gold-standard answer that may itself differ syntactically from an equally valid query.

A synthetic fintech SQLite database was constructed locally with four related tables (users, merchants, transactions, and loans) populated with realistic sample records, so that every generated query could be executed against real data rather than evaluated as inert text.

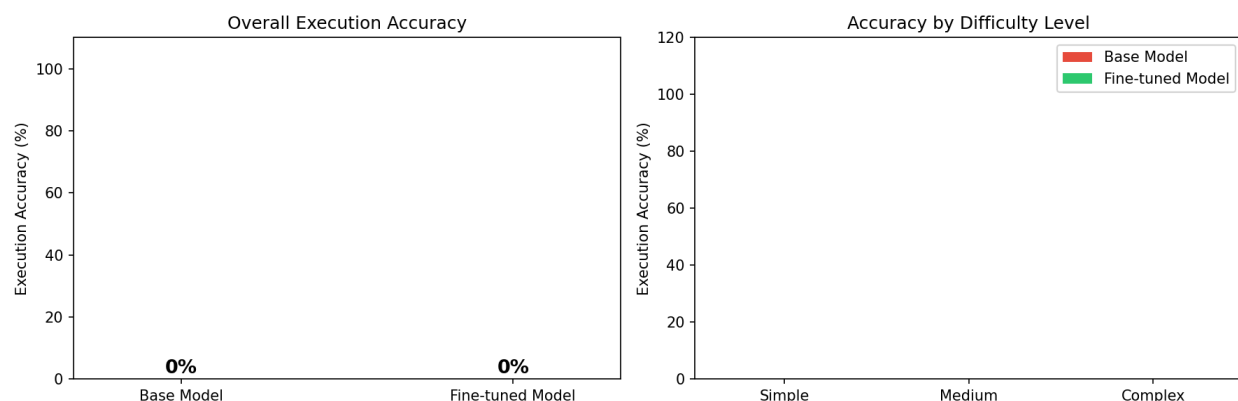
Ten natural language questions were authored across three difficulty tiers (three Simple, four Medium, three Complex), spanning single-table aggregation up to multi-table joins with grouping

and ordering. For each question, SQL was generated independently by both the unmodified base model and the fine-tuned model, under identical greedy decoding settings (temperature=0), and each output was executed via a single Python `sqlite3.Cursor.execute()` call inside a try/except block.

A specific behavior of Python's `sqlite3` module is central to the results below: `cursor.execute()` is a single-statement API. If the supplied string contains a terminated statement followed by any additional non-empty text including prose, `sqlite3` raises *"You can only execute one statement at a time."* rather than running the first statement and discarding the rest. This makes the execution harness strict by design: a model must emit exactly one clean SQL statement with nothing appended, mirroring the precision expected of a production text-to-SQL service.

4. Key Findings & Error Analysis

Both the base model and the fine-tuned model recorded a 0% execution success rate across all ten questions. Taken at face value this reads as an uninformative null result. Taken apart by model, it is not, the two models failed for two structurally different and individually diagnosable reasons, and isolating them was only possible because the evaluation was execution-based rather than similarity-based.



4.1 Base Model: Chatty-Formatting Failure

The base, instruction-tuned model's SQL reasoning was frequently sound. For several questions (e.g. "What is the average credit score of users in Jakarta?" and "Which users have active loans and what are their credit scores?") it produced a single, correctly-structured, schema-accurate SQL statement that would have executed successfully and returned correct results.

However, as an instruction-tuned chat model, it consistently continued generating past the terminating semicolon, appending explanatory prose under a self-generated *"### Explanation:"* or *"### Expected Output:"* heading. Under the strict single-statement execution harness described in Section 3, this trailing content caused every single base-model query, including the ones with structurally correct SQL, to be rejected outright with the same `sqlite3` error, regardless of whether the underlying query logic was right.

Root cause: the failure is a formatting/stopping-criteria problem, not a SQL-reasoning problem. The base model was never trained to stop at the SQL boundary, because it was never given a reason to, its instruction-tuning objective rewards explaining the answer, not just producing it.

4.2 Fine-Tuned Model: Structural Over-Complication

The fine-tuned model fully solved the formatting problem: across all ten outputs, it emitted zero conversational filler and respected the strict `#### SQL:` boundary perfectly. This confirms the prompt-template fine-tuning objective from Section 2.2 was learned correctly.

What replaced the formatting failure was a structural one. On simple, single-table questions, the model frequently produced unnecessarily complex SQL: UNION-ing unrelated tables that share no meaningful join key, joining on hallucinated columns that do not exist in the schema (e.g. `T2.merchants_id` where the schema defines `merchant_id`), and injecting filter conditions referencing values never present in the question or schema, such as a city ('London') and date threshold ('2015-01-01') that appeared nowhere in the prompt but recurred across multiple unrelated answers.

Root cause: this is a dataset-composition effect rather than a prompt-format issue. *sql-create-context*, like most general text-to-SQL corpora, skews toward multi-table, multi-condition queries, since those are the examples that best demonstrate schema reasoning. With only one epoch of exposure (itself a downstream consequence of the Section 2.4 checkpointing failure), the model appears to have anchored on structural patterns (join chains, UNIONS, multi-clause filters) as a stylistic prior for “what a SQL answer looks like,” independent of whether the specific question actually called for that structure. The recurrence of the identical hallucinated filter values across unrelated questions is consistent with this: it suggests pattern memorization of a frequent training-set shape rather than question-specific reasoning.

4.3 Synthesis

Read together, the two failure modes are close to complementary. The base model demonstrates that SQL reasoning ability was already present without any fine-tuning, but execution-readiness (format discipline) was not. The fine-tuned model demonstrates the inverse: format discipline was fully and successfully taught, but at the cost of the simple-query reasoning the base model had handled correctly, plausibly traded away in exchange for absorbing the dataset's structural complexity bias under a single epoch of training. Neither model is execution-ready as-is, but the two results together point at two specific, separately addressable engineering levers, covered in Section 5.

5. Conclusion & Productionizing Next Steps

The 0% headline execution rate should not be read as a dead end; it isolated two independent, fixable failure modes that a similarity-based evaluation would have masked. The following next steps follow directly from Section 4 and the infrastructure issues in Section 2.4.

5.1 Address fine-tuned-model dataset bias

- Rebalance the training mix to deliberately over-represent simple, single-table queries relative to their natural frequency in *sql-create-context*, to counteract the complexity skew identified in Section 4.2.
- Complete the originally planned 3 full epochs (once the checkpointing fix below is in place) to test whether additional exposure reduces over-complication, or whether it is purely a data-distribution effect that more training would not fix.
- Add a small set of adversarial “simple-question, simple-schema” examples specifically designed to penalize unnecessary joins/UNIONS during training.

5.2 Address base-model chatty-formatting failure

- Constrain generation with an explicit stopping criterion on the first semicolon, or post-process by truncating at the first statement boundary before execution, relevant for any deployment that might fall back to a non-fine-tuned model.
- If the production path always uses the fine-tuned model, this is lower priority; it remains useful context for understanding what fine-tuning actually changed.

5.3 Resolve the checkpointing instability

- Upgrade the trl / transformers / peft version set together (rather than pinned individually) to versions confirmed mutually compatible by unsloth-zoo's own dependency resolution, to eliminate the dual-import class-identity defect behind the PicklingError.
- Once stable, re-enable periodic checkpointing, ideally to persistent storage (e.g. Google Drive) rather than local ephemeral disk, to restore crash recovery without repeating the original failure.
- Avoid `model.merge_and_unload()` on 4-bit-quantized models until the underlying Unsloth/PEFT incompatibility is confirmed fixed upstream; the adapter-plus-base-model loading pattern used in the final inference pipeline is the safer default regardless.

Taken together, the architecture and method choices in this project were sound for the compute budget available, the two infrastructure failures were root-caused rather than worked around blindly, and the evaluation design surfaced two genuinely distinct, independently actionable failure modes rather than a single uninformative zero.